

AI in Dynamic Action

Team Project Requirements

Courses: Reinforcement Learning, Advanced Control Methods

Document authors: P. Osinenko, V. Geniavoi, S. Ibrahim

April 2026

1. General Requirements

- Students organize in teams of at most 5
- Projects are organized in topics. The topics are announced in the course and education platform (e.g., canvas). To each topic, each team chooses a concrete problem (control task or Markov decision process). The problem must be approved by the professor, co-instructor or TA. If you struggle to pick a problem, ask course instructors for suggestions
- Each project is reported via:
 - a code repository (in exceptional cases upon approval by a course instructor – an interactive Python notebook)
 - a video blog of max. 10 minutes at most featuring all team members about the project, highlighting the historical development of it. AI use for essential parts (visualization is non-essential) must be explained
- Project is defended via a Q&A session, 5 min. max

2. General Repository Rules

- Each team must create one repository for the whole team
- Each project must be placed in a separate folder named:
`project_<number>_<topic_name>_<system_name>`
Example:
`project_2_adaptive_control_acrobot`
- The repository must be clean, readable, and easy to navigate

3. Required Structure for Each Project

Each project folder must contain at least:

- `README.md`
- `src/`

- `figures/`
- `animations/`
- `configs/` or `params/`

Recommended code structure inside `src/`:

- `system.py` or `systems/...` or `plant.py` or `plants/...` (in Markov decision processes, usually “environment” instead of “system”)
- `controller.py` or `controllers/...` (in Markov decision processes, usually “agent” instead of “controller”)
- `simulation.py`
- `visualization.py`
- `main.py`

The implementation must be clearly separated into logical parts. In particular:

- the system / plant must be separated from the controller
- simulation code must not be mixed with plotting code
- visualization code must not be mixed with core dynamics or control logic

Do not submit a project where everything is placed in one large file or one notebook.

4. README Requirements

Each project must have a complete `README.md` that explains the project clearly.

4.1 Problem Definition

- Control/decision problem to be addressed
- Plant or environment definition
- Assumptions, context
- Class of methods to be applied (e.g., policy gradient, adaptive control)

4.2 System Description

- System / plant / environment description
- Drawing, animation or diagram of the system whenever appropriate
- State variables
- Control input (also called “action”) specification
- Unknown parameters, if applicable
- Control bounds / constraints
- Equations of state dynamics

4.3 Mathematical Specification

- All notation must be defined when first used
- All variables must be explained clearly
- No symbol reuse with different meanings in the same report is allowed
- Consistent notation across `README`, code, plots, and video must hold

Examples of bad practice:

- Using symbols such as τ , d , V , Q without defining them

- Reusing one letter for two different meanings
- Using vague terms without explanation

4.4 Method Description

- The control law / optimization method used
- The derivation idea, at least at a concise level
- Convergence / stability or other relevant formal analysis
- Derivation of essential parts of the method, e.g., adaption laws
- Approximations or simplifications that were made

4.5 Algorithm Listing

Each project must contain at least one of:

- pseudocode
- a structured algorithm block
- a short numbered pipeline describing the full method

It must be possible to understand:

- what is computed
- in what order
- from which signals
- using which equations or quantities

4.6 Experimental Setup

- initial conditions
- reference state or trajectory
- initial parameter estimates, if relevant
- simulation time
- controller parameters
- any disturbances, model mismatch, or uncertainty used in tests

4.7 Reproducibility

- exact commands to run the project
- exact commands to reproduce the final plots and animations
- dependency file (requirements.txt or equivalent)
- description of produced outputs

4.8 Results Summary

- short summary of what works
- short summary of what does not work or what remains limited
- brief interpretation of the final results

5. Theory and Implementation Must Match

The repository, README, plots, and video must describe the same method that is actually implemented.

This means:

- no claiming one method while the code uses another
- no hidden extra components that are not explained
- no misleading method names
- no mismatch between equations, notation, and implementation

If a simplified or modified version of the intended method is implemented, this must be stated explicitly.

6. Code Quality and Cleanliness

The project code must be readable and reasonably organized.

Requirements:

- clear file names
- no clutter from many abandoned versions of the same script
- no files such as `final_final2.py`, `new_last_real.py`, etc.
- no obviously personal local machine paths in the repository
- repeated code should be minimized where possible

Notebooks may be used for exploration, but the final project must be reproducible from proper scripts.

6. Required Results and Artifacts

Each project must include:

- at least one clear simulation result showing that the method works
- at least one animation (gif or mp4)
- a set of readable final plots
- short textual interpretation of the main results

At minimum, the result section should usually contain (control theory case):

- state trajectories over time
- control signal over time
- tracking error over time, where applicable
- parameter estimation / adaptation plots, where applicable
- phase plots, where applicable
- learning curves with statistical data, e.g., confidence bounds, in case of reinforcement learning projects

7. Plot Requirements

All plots must be self-explanatory. Consider informative captions with short conclusions. What should the reader actually conclude from the plot?

Each important plot must include:

- a title
- labeled axes
- units where applicable
- a legend if more than one curve is shown
- readable scale and font sizes
- a short caption or explanation nearby

The explanation must state:

- what exactly is being plotted
- what conclusion should be drawn from the plot
- what important effect or limitation is visible

Recommended good practice:

- show reference and actual trajectories clearly
- mark constraints if they matter
- mark initial and target states if that improves readability
- for trajectory plots, a time-gradient coloring can be useful where appropriate

Unacceptable plot practice:

- unlabeled curves
- unreadable scales
- plots without captions or explanation
- plots where it is impossible to understand what quantity is shown
- screenshots of plots instead of properly exported figures

9. Animation Requirements

Each project must include a clear and informative animation. The animation should make the system behaviour understandable, not merely decorative.

Whenever applicable, the animation should show:

- the system itself
- the target state or target trajectory
- the actual motion / response
- orientation, if relevant
- current time
- important quantities such as error or control input, if useful

A very minimal animation that does not help interpret the project is not sufficient.

10. Comparison Requirements

Project 1

Comparison is welcome but not mandatory. It is sufficient to show clearly that the chosen method works and to explain the obtained result well.

Projects 2 and Higher

Comparison is mandatory.

The comparison must clearly show how an alternative approach fails, underperforms, or becomes less convincing.

Valid alternatives may include:

- the previous project's method
- a simpler controller
- a controller without adaptation / robustness / planning
- no control
- a weaker baseline version of the same method

The comparison must be shown using plots and, where useful, animations.

Useful failure modes to demonstrate:

- divergence
- oscillations
- slow convergence
- steady-state error
- poor tracking
- instability under uncertainty
- violation of constraints
- weak behavior on more difficult initial conditions

11. Scope and Task Selection

Choose a task that is challenging but still realistic to finish well.

Teams may lose points if:

- the task is too ambitious for the available time
- the setup is unnecessarily complicated
- the project remains unfinished
- the team does not simplify the problem when the original setting does not work

A simpler but well-executed project is better than an overcomplicated unfinished one

12. Video Requirements

The video report must be clear, concise, and focused.

12.1 Mandatory Requirements

- maximum length: 10 minutes
- every team member must speak
- the narration must be understandable
- the sound volume must be normalized across speakers
- the sound quality must be good

12.2 Visual Cleanliness

The video must not contain unnecessary visual noise, including:

- uncut transitions

- accidental notifications
- irrelevant windows or desktop clutter
- duplicated fragments
- awkward empty pauses
- distracting montage artifacts

12.3 Content Requirements

The video must explain:

- the system
- the control / decision goal
- the chosen method
- the development history of the project
- failed ideas or weak intermediate versions
- the final design decisions
- how to read the plots
- what the animation demonstrates
- the final result and its limitations

For Projects 2 and higher, the video must also explain:

- how the previous method or alternative baseline fails
- what exactly was changed
- why the new method performs better

12.4 What to Avoid

- reading code for a long time without explanation
- showing formulas or plots without interpretation
- speaking too fast while skimming through implementation details
- overly rushed or overly long presentation
- omitting some team members from the report
- low-quality or highly uneven audio

13. Minimum Checklist Before Submission

Before submission, verify that your project answers all of the following:

1. Is the system clearly defined?
2. Is the control objective clearly defined?
3. Is all notation explained?
4. Is there a clear algorithm listing or method pipeline?
5. Do the theory, code, and README match?
6. Are the plots self-explanatory?
7. Are the main plots explained in words?
8. Is there a clear animation?
9. Is the repository clean and easy to navigate?
10. For Projects 2+, is there a convincing comparison showing why the alternative is weaker?
11. Does the video stay within the time limit and include all team members?
12. Is the video free from audio and visual clutter?

14. Summary

A strong project is not only technically correct. It is also:

- well structured
- reproducible
- mathematically clear
- visually readable
- honest about failed attempts and limitations
- convincing in both the repository and the video report

The goal is not just to show that something runs

The goal is to show that the team understands the system, the method, the implementation, and the obtained results